More practice - Binary search

2 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	1
EASY	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

More practice - Binary search | 2 questions | tap a title to jump to it

E 1. Check an Interval Against Existing Intervals for Conflicts **PracHub 2026-08-08**

H 2. Find Maximum Envelope Nesting **PracHub 2026-03-18**

STUDY PATH: question 127 of 167 (more practice)

Priority score 3.36 | Binary search | asked 73 days ago

EASY | **IN APP** **LC 729** | **PracHub** | **Technical Screen** | **1 of 2**

Check an Interval Against Existing Intervals for Conflicts

POSTED 2026-08-08 | 73 DAYS AGO | LAST 90 DAYS

STUDY: TALK **ALL TIME 26.5** **30 DAYS 0** **PRIORITY 132/228**

ABOUT THIS QUESTION

Topic: More practice - Binary search

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #729 My Calendar I

LeetCode match: LC 729 My Calendar I (inferred from overview)

Posted on PracHub: 2026-08-08 (updated 2026-08-29)

Original: https://prachub.com/coding-questions/check-an-interval-against-existing-intervals-for-conflicts

QUESTION

Overview: Check whether one half-open candidate interval conflicts with any interval in a large unsorted collection. Respect strict start and end semantics so ranges that only touch at an endpoint do not conflict.

APPROACH

Half-open intervals [s,e) conflict iff new.start < e and s < new.end (touching endpoints do not conflict). One-off check on an unsorted collection is a single O(n) scan; for repeated queries keep a sorted structure (TreeMap/balanced BST or sorted list + bisect) and check only the neighbors. O(log n) per query.

HOW TO THINK ABOUT IT

Two half-open intervals [s1, e1) and [s2, e2) overlap exactly when s1 < e2 and s2 < e1. If the booked intervals are kept sorted and disjoint, only the neighbours at the insertion point can conflict, so a binary search replaces the scan.

Basis: LeetCode 729, My Calendar I (booking succeeds only when the new half-open interval overlaps nothing), and 731 for double booking.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
booked = []
def book(s, e):
    for s2, e2 in booked:
        if s < e2 and s2 < e:
            return False  # overlap
    booked.append((s, e))
    return True
```

BRUTE FORCE

TIME	SPACE
O(n) per booking	O(n)

WHY IT IS SLOW

Each booking scans all earlier bookings, O(n). With the intervals sorted, binary search finds the only two neighbours that could conflict.

OPTIMAL SOLUTION · Sorted starts with bisect: check only two neighbours

TIME	SPACE
O(log n) search, O(n) list insert	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Allow double booking but never triple booking (LeetCode 731).

Keep a difference map (+1 at start, -1 at end) and sweep it in sorted order; if the running count ever exceeds 2, undo the change and reject. O(n log n) per booking, or use a segment tree for faster.

FOLLOW-UP 2 Report the maximum number of overlapping bookings after each booking (LeetCode 732).

The same difference map, taking the maximum running count in the sweep. A segment tree with lazy range increments gives O(log n) per booking.

FOLLOW-UP 3 Closed intervals instead of half-open.

Then touching endpoints conflict, so the tests become <= instead of <. Say it, and keep the choice in one comparison.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Check an Interval Against Existing
# Intervals for Conflicts
#
# Check whether one half-open candidate interval conflicts
# with any interval in a large unsorted collection.
# Respect strict start and end semantics so ranges that
# only touch at an endpoint do not conflict.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Allow double booking but never triple booking
# (LeetCode 731).
# 2. Report the maximum number of overlapping bookings
# after each booking (LeetCode 732).
```

```
# 3. Closed intervals instead of half-open.
#
# INTERVIEW NOTES
# Two half-open intervals [s1, e1) and [s2, e2) overlap
# exactly when s1 < e2 and s2 < e1. If the booked
# intervals are kept sorted and disjoint, only the
# neighbours at the insertion point can conflict, so a
# binary search replaces the scan.
#
# BRUTE FORCE (say it first): Scan every booked interval
# Compare the new interval with all existing ones using
# the overlap test. Correct and simple, but each booking
# costs time proportional to the number of bookings. Time
# O(n) per booking. Space O(n).


from bisect import bisect_right
from collections import defaultdict


# BRUTE FORCE (say it first): Scan every booked interval |
# time O(n) per booking
class CalendarBrute:
    def __init__(self):
        self.booked = []

    def book(self, start, end):
        if any(
            start < e and s < end
            for s, e in self.booked
        ):
            return False
```

```
        self.booked.append((start, end))
        return True


# BASE QUESTION (optimal): Sorted starts with bisect:
# check only two neighbours
# Idea: Keep the booked intervals sorted by start.
class Calendar:
    """A sorted calendar: O(log n) per check."""

    def __init__(self):
        self.starts, self.ends = [], []

    def book(self, start, end):
        i = bisect_right(self.starts, start)
        # the booking before must end by `start`
        if i > 0 and self.ends[i - 1] > start:
            return False
        # the booking after must start at or after `end`
        if i < len(self.starts) and self.starts[i] < end:
            return False
        self.starts.insert(i, start)
        self.ends.insert(i, end)
        return True


# FOLLOW-UP 1: Allow double booking but never triple
# booking (LeetCode 731).
# Answer: Keep a difference map (+1 at start, -1 at end)
# and sweep it in sorted order; if the running count ever
# exceeds 2, undo the change and reject. O(n log n) per
# booking, or use a segment tree for faster.
class DoubleBooking:
    """LeetCode 731: two overlaps are fine, three are not.
    LeetCode 732: also report the largest overlap."""

    def __init__(self, limit=2):
        self.delta = defaultdict(int)
        self.limit = limit
```

```
    def peak(self):
        # sweep the +1 / -1 changes in time order
        run = best = 0
        for t in sorted(self.delta):
            run += self.delta[t]
```

```
            best = max(best, run)
        return best

    def book(self, start, end):
        self.delta[start] += 1
        self.delta[end] -= 1
        if self.peak() > self.limit:
            # undo the change and reject the booking
            self.delta[start] -= 1
            self.delta[end] += 1
            return False
        return True


# HELPERS for the checks below
def conflicts(candidate, intervals):
    # half-open [start, end): touching ends do NOT
    # conflict
    s, e = candidate
    return any(a < e and s < b for a, b in intervals)


if __name__ == "__main__":
    taken = [(10, 20), (30, 40)]
    assert conflicts((15, 25), taken)
    assert not conflicts((20, 30), taken)
    assert conflicts((19, 31), taken)
    # sorted calendar
    c = Calendar()
    assert c.book(10, 20) and c.book(20, 30)
    assert not c.book(15, 25) and c.book(5, 10)
    # double booking is allowed, triple booking is not
    d = DoubleBooking()
    assert d.book(10, 20) and d.book(15, 25)
    assert not d.book(17, 18)
    assert d.peak() == 2
    # limit 1 is the plain no-overlap calendar
    one = DoubleBooking(limit=1)
    assert one.book(1, 5) and not one.book(4, 6)
```

FUNCTION AND TEST CASES

Signature

```
def hasIntervalConflict(candidate: list[int], intervals: l
ist[list[int]]) -> bool:
```

Sample call

```
hasIntervalConflict([4,7], [[1,3],[7,9]])
```

Test cases

Input	Expected
([4,7],[[1,3],[7,9]])	False
([4,7],[[1,5],[8,9]])	True

STUDY PATH: question 128 of 167 (more practice)

Priority score 3.11 | Binary search | asked 216 days ago

HARD | IN APP LC 354 | PracHub | Technical Screen | 2 of 2

★ TOP 75: rank 14 in the Medium and Hard priority list

★ Find Maximum Envelope Nesting

POSTED 2026-03-18 | 216 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 52.8 | 30 DAYS 33 | PRIORITY 27/228

ABOUT THIS QUESTION

Topic: More practice - Binary search

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #354 Russian Doll Envelopes

LeetCode match: LC 354 Russian Doll Envelopes

Posted on PracHub: 2026-03-18 (updated 2026-05-06)

Original:
https://prachub.com/coding-questions/find-maximum-envelope-nesting

QUESTION

Overview: This question evaluates algorithmic problem-solving skills, particularly reasoning about multi-dimensional ordering, sequence optimization, and time-complexity analysis.

APPROACH

Sort by width asc and height desc on ties, then find the strictly increasing subsequence on heights with binary-search LIS (patience sorting). O(n log n).

HOW TO THINK ABOUT IT

Two dimensions are hard, one is easy. Sort by width so that width order is settled, then the question becomes a longest strictly increasing subsequence of heights. The trap is envelopes with equal width: they cannot nest, so sort equal widths by height descending, which prevents two of them from forming an increasing pair. The LIS is then done with patience sorting (binary search over tails).

Basis: LeetCode 354, Russian Doll Envelopes (strictly larger in both width and height).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
envelopes.sort()  # by width
best = [1] * len(envelopes)
for i in range(len(envelopes)):
    for j in range(i):
        w1, h1 = envelopes[j]
        w2, h2 = envelopes[i]
        if w1 < w2 and h1 < h2:
            best[i] = max(best[i], best[j] + 1)
answer = max(best)
```

BRUTE FORCE

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Every envelope compares with all earlier ones, O(n^2). Sorting by width ascending and height descending, then a longest increasing subsequence on heights with binary search, is O(n log n).

OPTIMAL SOLUTION · Sort by width ascending and height descending, then LIS by binary search

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the actual chain of envelopes.

Remember, for each element, the index of the tail it extended (parent pointers), and rebuild from the last tail of the longest length. The sorted order and binary search are unchanged.

FOLLOW-UP 2 Three dimensions (boxes).

Sorting plus LIS no longer reduces the problem; use an O(n^2) DP, or CDQ divide and conquer with a Fenwick tree for O(n log^2 n).

FOLLOW-UP 3 Non-strict nesting (equal sizes allowed).

Sort by width ascending and height ascending, and use bisect_right (longest non-decreasing subsequence) instead of bisect_left.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find Maximum Envelope Nesting
#
# This question evaluates algorithmic problem-solving
# skills, particularly reasoning about multi-dimensional
# ordering, sequence optimization, and time-complexity
# analysis.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the actual chain of envelopes.
# 2. Three dimensions (boxes).
# 3. Non-strict nesting (equal sizes allowed).
#
# INTERVIEW NOTES
# Two dimensions are hard, one is easy. Sort by width so
# that width order is settled, then the question becomes a
# longest strictly increasing subsequence of heights. The
# trap is envelopes with equal width: they cannot nest, so
```

```
# sort equal widths by height descending, which prevents
# two of them from forming an increasing pair. The LIS is
# then done with patience sorting (binary search over
# tails).
#
# BRUTE FORCE (say it first): DP over pairs: longest chain
# ending at each envelope
# Sort by width, then best[i] is 1 plus the best best[j]
# over earlier envelopes that are strictly smaller in both
# dimensions. Correct and the oracle; quadratic. Time
# O(n^2). Space O(n).


from bisect import bisect_left, bisect_right


# BRUTE FORCE (say it first): DP over pairs: longest chain
# ending at each envelope | time O(n^2)
def nesting_dp(envelopes):
    env = sorted(envelopes)
    best = [1] * len(env)
    for i in range(len(env)):
        for j in range(i):
            if (
                env[j][0] < env[i][0]
                and env[j][1] < env[i][1]
            ):
                best[i] = max(best[i], best[j] + 1)
    return max(best, default=0)


# BASE QUESTION (optimal): Sort by width ascending and
# height descending, then LIS by binary search
# Idea: Sorting equal widths by decreasing height means
# two envelopes of the same width can never both appear in
# a strictly increasing height sequence.
def max_envelopes(envelopes):
    # sort by width, taller first on ties, then find the
    # longest
    # strictly increasing run of heights
    ordered = sorted(envelopes,
                     key=lambda e: (e[0], -e[1]))
    tails = []
    for _, h in ordered:
        i = bisect_left(tails, h)
        if i == len(tails):
            tails.append(h)
        else:
            tails[i] = h
    return len(tails)


# FOLLOW-UP 1: Return the actual chain of envelopes.
# Answer: Remember, for each element, the index of the
# tail it extended (parent pointers), and rebuild from the
# last tail of the longest length. The sorted order and
# binary search are unchanged.
def envelope_chain(envelopes):
    # also return the chain: remember what each element
    # extended
    ordered = sorted(envelopes,
                     key=lambda e: (e[0], -e[1]))
    tails, tail_idx = [], []
    prev = [-1] * len(ordered)
    for i, (_, h) in enumerate(ordered):
        k = bisect_left(tails, h)
        if k == len(tails):
            tails.append(h)
            tail_idx.append(i)
        else:
            tails[k] = h
            tail_idx[k] = i
        # the element that ended a chain one shorter
        prev[i] = tail_idx[k - 1] if k else -1
    chain = []
    i = tail_idx[-1] if tail_idx else -1
    while i != -1:
        chain.append(ordered[i])
        i = prev[i]
    return chain[::-1]


# FOLLOW-UP 2: Three dimensions (boxes).
# Answer: Sorting plus LIS no longer reduces the problem;
# use an O(n^2) DP, or CDQ divide and conquer with a
# Fenwick tree for O(n log^2 n).
def max_boxes_3d(boxes):
    # three sizes: the reduction no longer works, use
    # O(n^2)
    ordered = sorted(boxes)
    best = [1] * len(ordered)
    for i in range(len(ordered)):
        for j in range(i):
            a, b = ordered[j], ordered[i]
            if all(x < y for x, y in zip(a, b)):
                best[i] = max(best[i], best[j] + 1)
    return max(best, default=0)


# FOLLOW-UP 3: Non-strict nesting (equal sizes allowed).
# Answer: Sort by width ascending and height ascending,
# and use bisect_right (longest non-decreasing
# subsequence) instead of bisect_left.
def max_nested_non_strict(envelopes):
    # equal sizes may nest: sort both ways ascending and
    # use
    # bisect_right (a longest non-decreasing subsequence)
    tails = []
    for _, h in sorted(envelopes):
        i = bisect_right(tails, h)
        if i == len(tails):
            tails.append(h)
        else:
            tails[i] = h
    return len(tails)


if __name__ == "__main__":
```

```
    envs = [[5, 4], [6, 4], [6, 7], [2, 3]]
    assert max_envelopes(envs) == 3
    chain = envelope_chain(envs)
    assert len(chain) == 3
    # every envelope fits strictly inside the next
    for a, b in zip(chain, chain[1:]):
        assert a[0] < b[0] and a[1] < b[1]
    assert envelope_chain([]) == []
    # equal sizes: only the non-strict version nests them
    twins = [[3, 3], [3, 3], [4, 4]]
    assert max_envelopes(twins) == 2
    assert max_nested_non_strict(twins) == 3
    boxes = [(1, 1, 1), (2, 2, 2), (3, 1, 4), (3, 3, 3)]
    assert max_boxes_3d(boxes) == 3
```

FUNCTION AND TEST CASES

Signature

```
def solution(envelopes: list[list[int]]) -> int:
SAMPLE INPUT
[[5, 4], [6, 4], [6, 7], [2, 3]]
```

Test cases

Input	Expected
`[[5, 4], [6, 4], [6, 7], [2, 3]]`	`3`
`[]`	`0`

ALSO REPORTED · 1 more

Russian Doll Envelopes | 1Point3Acres | phone-screen | Single report

https://www.1point3acres.com/interview/problems/company/google/russian-doll-envelopes

Sorting heights descending on width-ties prevents same-width envelopes from chaining — easy to miss.
LIS via bisect_left on a tails array is the standard O(n log n) implementation.

Preparation: Write LIS O(n log n) from memory in under 8 min.
Justify the descending-height-on-tie trick out loud.
Plain LIS and the mountain-LIS variant make good drill partners.